

Beherrschungsstrats gegen Komplexität

1. Abstraktion: geeignete Modelle, ohne Details
2. Spezialisierung: Aufteilung in Unterstrukturen
3. Standardisierung: Dokumentation, Wiederverwendung
4. Wktze: Präzision, Leistungsfähigkeit

Softwareanprüche

korrekt, effizient, lesbar, zuverlässig, benutzbar, erweiterbar, sicher, wartbar, wiederverwendbar
Entwicklungsprozess

Anforderungen

Initial: Anf. an Verhalten
 Quel: Bezug auf Qualitätsmerkmal
 RBS: eingeschränkter Lsg.raum

Qualitätssicherung

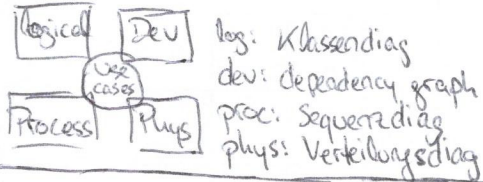
Indiv. Anf: korrekt, eindeutig, umsetzbar, prüfbar, nachverfolgbar

Anf. Mengen: minimal, konsistent, vollständig, anpassbar

Architektur

S ingle responsibility: 1 Schnittstelle → 1 Aufgabe
 O pen-closed: neue Funkt.alität erlauben, vor Änderung schützen
 L istroy-Subst: zwei Kompos mit gleicher Schnittstelle austauschbar
 I nterface Segregation: so wenig Abhäng. wie möglich
 D ependency Inversion: high-level Modules unabh. von low-level

4+1 Sichten



CAP-Theorem

network partitions:
 consistency + availability not at the same time

Architekturmuster

Client-Server: C muss S kennen einfache Verteilung v. Daten
 OO-Arch: Kapselung d. Kommunikation
 Obj. müssen andere Obj. kennen
 Schichten: hori. Decomp: Schicht bietet drüberliegenden Schicht Funkt.alität
 geschlossen vs open
 vert. Decomp: alle Features in Proc unabh. von anderen Procs

Kopplung schwach, Kohäsion steile, wartbar, weniger Code

Abstrakte Syntax

$C = \langle i, s, o, l, \Gamma \rangle$ mit
 $i = \{i, j\}$, $s = \{s_1, s_2\}$, $o = \{o, j\}$, $l = \{l_1, l_2\}$
 $\Gamma = \{s_1 \rightarrow off \wedge \neg i \wedge s_2 \rightarrow off \wedge o = s_1\} \vee \dots$

Transitionen

$\{s_1 \rightarrow off\} \xrightarrow{\{i, j \rightarrow o\} / \{o, j \rightarrow o\}} \{s_2 \rightarrow off\} \dots$

Lauf

Var to t_1, t_2, t_3, t_4
 i_1 0 0 1 1 0
 s_1 off off off on on
 s_2 off off on on on
 o_1 off off on on on

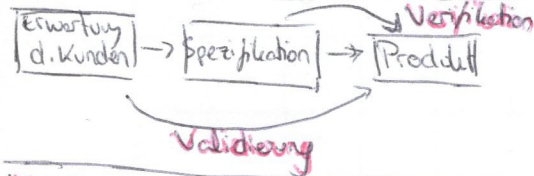
true positive: P. verdetzt, F. liegt vor
 true negative: P. unverdetzt, F. nicht
 false positive: P. unverdetzt, F.
 false negative: P. verdetzt, F. nicht

Softwarequalität

1. flücht: korrekt, sicher
2. zuverlässig: reif, fehlerlos
3. effizient: wirtschaftl.
4. wartbar: testbar, stabil
5. übertragbar: austauschbar
6. benutzbar: bedienbar, verständl.

Qualitätssicherung

produktorientiert: Software auf Quali. überprüfen
 prozessorientiert: Methoden, Wktze, Standards für Softwareerstellung

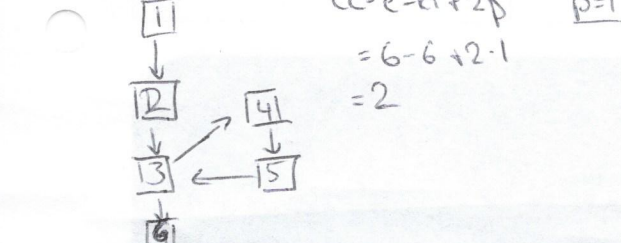


valid: richtig gebaut?
 valid: richtiges Produkt?

Metriken

LOC, ELOC, VLOC
 $LCOM = \frac{m - (\frac{1}{2} \sum_{i=1}^n n(A_i))}{m-1}$
 n Anz. Meths auf a
 a Anz. Felder
 m Anz. Meths

CFG



PERT

2-Plat: $E = \sum \frac{a+b}{2}$
 3-Plat: $E = \sum \frac{a+r \cdot c+b}{3}$
 $S = \frac{b-a}{6}$ $V = \left(\frac{b-a}{6}\right)^2$
 $r = 4$ $u = 6$

Risiko

$R = P \cdot S / R_i = P_i \cdot S_i$

Baule Baum

